CSC_3S005_EP Project Report: Heat Equation in CUDA

Genrikh Kuznetsov

June 6, 2026

1 Introduction

The objective of this project is to implement a numerical solver for the two-dimensional heat equation and compare a sequential CPU implementation with a CUDA GPU implementation. The heat equation describes the diffusion of heat over time. In this project, I considered a square two-dimensional grid with an initial hot region in the centre. The code simulates how the heat spreads over time.

The project was implemented in C++ and CUDA. The CPU version is used as a sequential benchmark, while the CUDA version parallelises the computation by assigning one GPU thread to each grid cell. The goal is to check that both versions produce the same numerical behaviour and to measure the performance gain obtained with the GPU implementation.

The repository is available at:

`https://github.com/genrikhkuz/csc305-heat-equation-cuda`

2 Numerical Scheme

The two-dimensional heat equation is

$$\frac{\partial U}{\partial t} = \frac{\partial^2 U}{\partial x^2} + \frac{\partial^2 U}{\partial y^2}.$$

I used the explicit finite-difference scheme given in the project document from Moodle:

$$U_{i,j}^{n+1} = (1 - 4\lambda)U_{i,j}^{n} + \lambda \left(U_{i+1,j}^{n} + U_{i-1,j}^{n} + U_{i,j+1}^{n} + U_{i,j-1}^{n} \right),$$

where

$$\lambda = \frac{\Gamma}{\Delta^2}.$$

In every run, I used

$$\lambda = 0.1.$$

This satisfies the stability condition required for the explicit scheme.

The boundary of the grid is kept fixed at zero which means that the code updates only the interior points of the grid, while the boundary points remain equal to zero at every time step. The initial condition is a square hot region placed at the center of the grid, with temperature 100.

3 Sequential CPU Implementation

The sequential implementation is written in C++. The two-dimensional grid is stored as a one-dimensional vector. A grid point (i, j) is stored at position

$$iN + j,$$

where N is the side length of the square grid.

The CPU implementation uses two grids:

- `current`, containing the temperature values at the current time step.
- `next`, containing the values computed for the next time step.

At each time step, the program loops over all interior grid points. For each point, it reads the four neighbouring values and applies the explicit finite-difference formula. After one time step is completed, the two grids are swapped. This avoids overwriting values that are still needed during the computation.

The CPU version also exports CSV files for visualisation and prints basic validation information such as minimum temperature, maximum temperature, total heat, and runtime.

For visualisation, I used a short Python script with `pandas` and `matplotlib`. The C++ and CUDA programs export the grid values to CSV files with columns for the coordinates and temperature. The Python script reads these CSV files, reconstructs the two-dimensional temperature grid, and displays it as a heat map. This visualisation code is not part of the numerical solver itself as it is only used to look at the results and produce the figures included in the report.

4 CUDA Implementation

The CUDA implementation follows the same numerical scheme as the CPU version. The main difference is that the update of the grid is parallelised on the GPU.

The CUDA kernel assigns one thread to one grid cell. Each thread computes its own coordinates using the CUDA block and thread indices:

$$j = \texttt{blockIdx.x} \cdot \texttt{blockDim.x} + \texttt{threadIdx.x},$$

$$i = \texttt{blockIdx.y} \cdot \texttt{blockDim.y} + \texttt{threadIdx.y}.$$

If the thread corresponds to an interior cell, it applies the heat equation update formula. If the thread corresponds to a boundary cell, it sets the value to zero.

The implementation uses two arrays on the GPU, corresponding to the current and next grids. At the beginning, the initial condition is copied from the CPU to the GPU using `cudaMemcpy`. Then the kernel is launched repeatedly for the required number of time steps. After each kernel launch, the two device pointers are swapped. At the end, the final grid is copied back to the CPU and can be saved as a CSV file.

The CUDA implementation uses the same tools as seen in the TDs:

- `cudaMalloc` for GPU memory allocation.
- `cudaMemcpy` for transfers between host and device.

- CUDA kernel launches.
- `cudaDeviceSynchronize` for synchronisation.
- `cudaFree` for freeing GPU memory.

The default block size is 16×16, corresponding to 256 threads per block. I also tested other block sizes, such as 8×8 and 32×8, to see how the runtime changes.

5 Validation

The numerical behaviour was first checked visually. The initial condition is a hot square in the centre of the domain. As time evolves, the heat spreads outward and the temperature field becomes smoother, which is the expected behaviour of the heat equation.

The following figures show the evolution of the heat distribution.

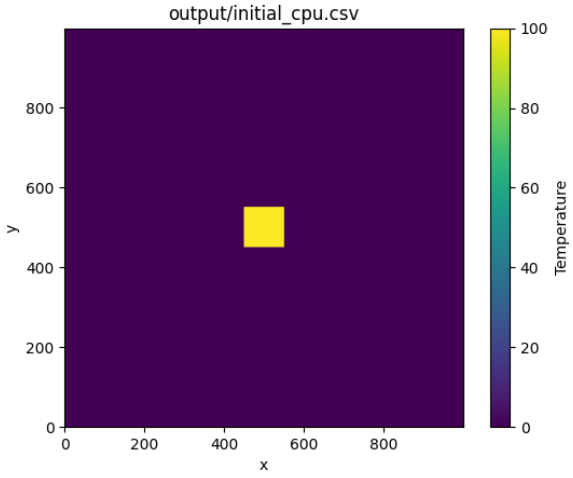

Figure 1: Initial heat distribution. The hot square is placed at the centre of the grid.

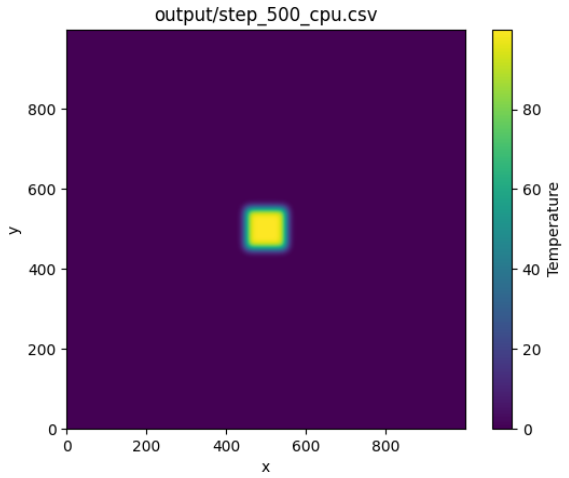

Figure 2: Intermediate CPU result after 500 time steps. The heat has started to diffuse outward.

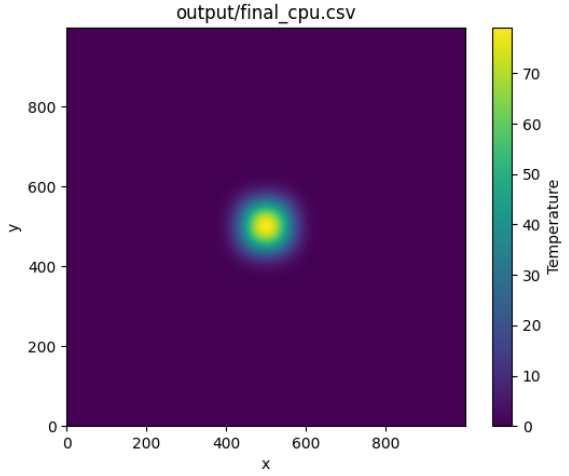

Figure 3: Final CPU result. The temperature distribution has become smoother.

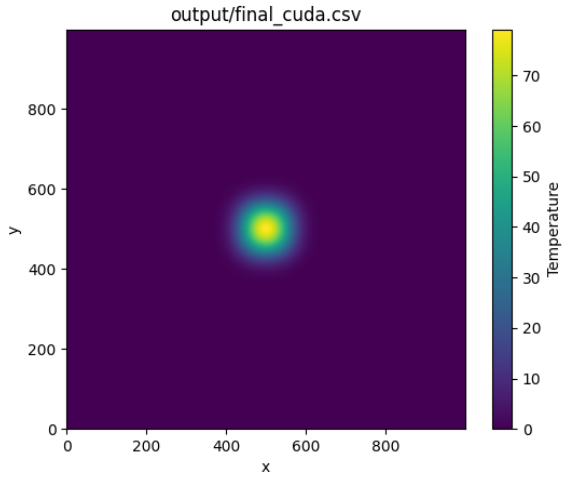

Figure 4: Final CUDA result. The result is visually identical to the CPU result.

The CPU and CUDA final visualisations are visually identical. This supports the correctness of the CUDA implementation.

I also compared basic numerical quantities printed by the two programs. For the large case with $N = 1000$, 5000 time steps, and radius 50, both the CPU and CUDA versions gave:

- minimum temperature: 0.
- maximum temperature: 79.1632.
- total heat: 1.0201×10^6.

The equality of these values shows that the CUDA version computes the same result as the sequential CPU version.

6 Benchmark Setup

The benchmarks were run on the machines in Salle info. The GPU information from `nvidia-smi` was:

- GPU: NVIDIA RTX A5000.
- GPU memory: 24564 MiB.
- driver version: 595.80.
- CUDA version: 13.2.

The CUDA code was compiled using:

```
/usr/local/cuda/bin/nvcc src/heat_cuda.cu -o heat_cuda \
    -arch=sm_60 -std=c++11 -I/usr/local/cuda/include
```

The CPU code was compiled using:

```
g++ -O2 -std=c++17 src/heat_cpu.cpp -o heat_cpu
```

For the performance benchmarks, CSV output was disabled. This was done to measure the computation time rather than file-writing time. The runtime reported is therefore the runtime of the simulation loop.

The speedup is computed as

$$\text{speedup} = \frac{\text{CPU runtime}}{\text{CUDA runtime}}.$$

7 Benchmark Results

7.1 Different Grid Sizes

The first benchmark compares the CPU and CUDA versions for three grid sizes. The CUDA block size is fixed at 16×16.

Case	Grid size	Steps	Block size	CPU time (s)	CUDA time (s)	Speedup
Small	300×300	1000	16×16	0.109408	0.0304386	3.59
Medium	500×500	2000	16×16	0.505107	0.0653275	7.73
Large	1000×1000	5000	16×16	6.06733	0.644260	9.42

Table 1: CPU and CUDA runtimes for different grid sizes.

The CUDA version is faster in all three cases. The speedup becomes larger as the grid size increases. This is expected because the GPU is better used when there is more parallel work. For the smallest case, the overhead of launching CUDA kernels and managing GPU execution is still relatively important compared with the total amount of computation. For the larger cases, there are many more grid cells and time steps, so the GPU can exploit more parallelism.

7.2 Effect of CUDA Block Size

I also tested different CUDA block sizes for the large case, with $N = 1000$, 5000 time steps, and radius 50.

Grid size	Steps	Block size	CUDA grid size	CUDA time (s)	Speedup
1000×1000	5000	8×8	125×125	0.446250	13.60
1000×1000	5000	16×16	63×63	0.498605	12.17
1000×1000	5000	32×8	32×125	0.471566	12.87

Table 2: Effect of CUDA block size on runtime for the large benchmark. The CPU reference time is 6.06733 seconds.

The best runtime among these tests was obtained with block size 8×8. The difference between the three block sizes is not so large, but it shows that the block configuration can affect runtime. The 16×16 block size remains a reasonable default because it is simple and gives 256 threads per block, but in this particular run 8×8 was faster.

8 Discussion

The explicit heat equation scheme is very suitable for GPU parallelisation because each grid cell at the next time step can be computed independently from the previous grid. This means that there is no need for synchronisation between threads inside one time step, except for the implicit synchronisation between successive kernel launches.

The implementation uses global memory only. Each thread reads five values from global memory: the centre value and the four neighbouring values. It then writes one value to the next grid. This is simple and close to the CUDA examples seen in the TDs.

The CPU implementation is purely sequential and is used as the reference version. I did not implement a multi-threaded CPU version. Therefore, the main comparison in this report is between sequential CPU execution and CUDA GPU execution. The different GPU block sizes correspond to different thread organisations on the GPU.

The numerical checks show that the CPU and CUDA implementations produce the same results for the tested cases. The minimum temperature remains zero, the maximum temperature decreases compared with the initial value 100, and the final heat maps show smooth diffusion from the center.

9 Task Repartition

I worked alone on this project.

The following tasks were, therefore, all done by me:

- implementation of the sequential CPU version.
- implementation of the CUDA version.
- CSV output and Python visualisation.
- correctness checks and comparison between CPU and CUDA.
- benchmark collection.
- writing of the report.

10 Conclusion

In this project, I implemented a sequential and a CUDA version of a two-dimensional heat equation solver. The numerical method is the explicit finite-difference scheme from the project document on Moodle. The CPU implementation computes the solution sequentially, while the CUDA implementation assigns one GPU thread to each grid cell.

The visualisations show the expected diffusion behavior: the initial hot square spreads outward and becomes smoother over time. The CPU and CUDA outputs match visually and numerically for the tested cases.

The performance results show a clear advantage for the CUDA version. For the largest benchmark with a 1000×1000 grid and 5000 time steps, the CPU version took around 6.07 seconds, while the CUDA version took less than one second. Depending on the block size, the observed speedup ranged from about $9.4\times$ to $13.6\times$.

The project therefore confirms that this explicit heat equation scheme is well suited for GPU parallelisation.

11 LLM Usage

Throughout the project, I have used LLM for two things:

Firstly, as I was unfamiliar with GitHub and Git, I seeked help from LLM in order to create, organise, and update the repository by asking it for terminal commands in git.

Secondly, as I was also quite unfamiliar with LaTeX, I seeked help with formatting i.e. creating the report layout, packages, asking how to make tables, add figures, add bullet lists, and so on.